

- 1. Structure du projet (Next.js 15 App Router)
- 2. Types et données (data/ et types/)
 - types/content.ts
 - data/siteContent.ts → refactorisé en data/siteContent/
 - data/mockUsers.ts
 - data/demoSeed.ts
- 3. État global et persistance
 - lib/demoStorage.ts — La couche « base de données » locale
 - context/DemoStoreContext.tsx — Le cœur de l'application
- 4. Authentification et rôles
 - components/PainterGate.tsx et components/AdminGate.tsx
 - components/RequireRole.tsx
- 5. Les shells (mise en page)
 - components/PublicShell.tsx
 - components/PainterShell.tsx
 - components/ControlShell.tsx
- 6. Les vues principales
 - views/AuthPage.tsx — Connexion + Inscription
 - views/DashboardPeintrePage.tsx — Tableau de bord peintre
 - views/AdminPage.tsx — Dashboard admin
 - views/AdminPeintresPage.tsx
 - views/AdminFichesPage.tsx / AdminRoadmapsPage.tsx
 - views/FichesPage.tsx / FicheDetailPage.tsx
 - views/RoadmapsPage.tsx / RoadmapDetailPage.tsx
 - views/HomePage.tsx
- 7. Les stubs (non fonctionnels)
- 8. Flux de données (comment tout s'assemble)
- 9. Cycle de vie d'une fiche

Voici une explication détaillée complète du code, organisée par couches architecturales.

1. Structure du projet (Next.js 15 App Router)

```
Frontend/src/
├─ app/                # Routes et pages (App Router)
│  ├─ page.tsx         → / (accueil)
│  ├─ fiches/          → /fiches, /fiches/[slug]
│  ├─ roadmaps/        → /roadmaps, /roadmaps/[slug]
│  ├─ categories/      → /categories
│  ├─ techniques/      → /techniques
│  ├─ oeuvres/         → /oeuvres
│  ├─ connexion/       → /connexion (auth)
│  ├─ dashboard/peintre/ → /dashboard/peintre
│  └─ admin/           → /admin, /admin/fiches, etc.
├─ components/        # Composants UI réutilisables
├─ context/           # React Context (état global)
├─ data/              # Données mockées (fiches, users, etc.)
├─ lib/              # Utilitaires (storage, slugify, API stub)
├─ types/             # Types TypeScript partagés
├─ views/             # Composants "page" (logique métier)
└─ middleware.ts      # Stub d'interception
```

Chaque dossier dans `app/` correspond à une route. Par exemple `app/fiches/[slug]/page.tsx` génère la route `/fiches/nom-de-la-fiche`.

2. Types et données (`data/` et `types/`)

`types/content.ts`

Définit les types de base :

- **FicheSection** — une section de contenu avec `label`, `title`, `paragraphs[]`
- **Fiche** — une fiche technique avec `slug`, `title`, `category`, `tags`, `sections[]`, etc.
- **Roadmap** — un parcours pédagogique avec `steps[]` contenant des `ficheSlugs[]`
- **DemoFiche** / **DemoRoadmap** — version étendue avec `authorId`, `status` (draft/review/published/rejected), `createdAt`, `updatedAt`
- **FicheFormInput** / **RoadmapFormInput** — types pour les formulaires

`data/siteContent.ts` → refactorisé en `data/siteContent/`

Les 28 fiches mockées ont été extraites dans un dossier dédié :

```
siteContent/  
├─ index.ts                # Point d'entrée – réexporte tout + assemble le  
tableau `fiches[]`  
├─ types.ts                # Types Fiche, Roadmap, etc.  
├─ meta.ts                 # publicNav, heroStats, adminMetrics  
├─ helpers.ts              # findFiche(), findRoadmap(), getRelatedFiches()  
├─ roadmaps.ts             # Les 4 roadmaps mockées  
├─ themeVariants.ts        # 4 thèmes de couleur  
└─ fiches/  
    ├─ fiche-glacis.ts     # ✅ déjà riche (9 sections)  
    ├─ fiche-impasto.ts    # ✅ déjà riche (7 sections)  
    ├─ fiche-sfumato.ts    # ✅ déjà riche (5 sections)  
    ├─ ...  
    └─ fiche-frottis.ts    # ❌ à enrichir (2 sections seulement)
```

data/mockUsers.ts

- **DemoUser** : { id, email, password, name, handle, role, bio, city, expertise, specialties[] }
- **DemoSession** : ce qui est stocké en session (sans password)
- **ProfileData** : champs modifiables du profil
- Un seul admin pré-défini : `admin@artplastique.demo`
- Fonctions : `findDemoUser()`, `toSession()`, `createDefaultSession()`, `isAdminEmail()`

data/demoSeed.ts

Crée l'état initial des fiches et roadmaps au premier chargement :

- `createInitialDemoData()` → 28 fiches seed (autrice "marie", statut "review" pour la première, "published" pour les autres) + 4 roadmaps seed + 1 fiche brouillon + 1 roadmap brouillon
- `mergeStoredDemoData()` → fusionne les données localStorage avec l'initial si localStorage vide

3. État global et persistance

lib/demoStorage.ts — La couche « base de données » locale

Stocke tout dans `localStorage` via 3 clés :

Clé	Contenu
<code>artplastique-demo-session</code>	<code>DemoSession</code> (utilisateur connecté)
<code>artplastique-demo-data</code>	<code>DemoData</code> (fiches + roadmaps)
<code>artplastique-registered-users</code>	<code>DemoUser[]</code> (peintres inscrits + seed fictifs)

Fonctions clés : `loadSession()`, `saveSession()`, `loadDemoData()`, `saveDemoData()`, `loadRegisteredUsers()`, `findRegisteredUser()`, `upsertRegisteredUser()`, `seedFictionalPainters()`.

context/DemoStoreContext.tsx — Le cœur de l'application

C'est un **React Context** qui fournit à tous les composants l'état global et les opérations :

```
const value = {
  session,           // utilisateur connecté ou null
  fiches,            // toutes les fiches
  roadmaps,          // toutes les roadmaps
  publishedFiches,    // fiches.status === 'published'
  publishedRoadmaps, // roadmaps.status === 'published'
  login(),            // authentication
  logout(),           // déconnexion
  register(),         // inscription nouveau peintre
  updateProfile(),    // modification du profil
  getFicheBySlug(),
  getRoadmapBySlug(),
  getMyFiches(),
  getMyRoadmaps(),
  getModerationQueue(),
  getRoadmapModerationQueue(),
  saveFicheDraft(),
  submitFicheForReview(),
  approveFiche(),
  rejectFiche(),
  deleteFiche(),
```

```
deleteRoadmap(),
saveRoadmapDraft(),
submitRoadmapForReview(),
approveRoadmap(),
rejectRoadmap(),
getAllRegisteredUsers(),
}
```

Fonctionnement :

1. Au démarrage (`useEffect`) : `seedFictionalPainters()` → charge session + données
2. `login(email, password)` : cherche d'abord dans l'admin pré-défini, puis dans `registeredUsers` (localStorage)
3. `register(name, email, password)` : crée un nouvel utilisateur peintre, le sauvegarde dans `registeredUsers`, connecte automatiquement
4. `updateProfile()` : modifie la session + synchronise dans `registeredUsers` pour persister après logout
5. `deleteFiche()` / `deleteRoadmap()` : supprime via `persistData()`

4. Authentification et rôles

`components/PainterGate.tsx` et `components/AdminGate.tsx`

Ces composants protègent les routes :

```
// PainterGate – pour /dashboard/peintre
if (!session) redirect('/connexion')
if (session.role !== 'peintre') redirect('/')

// AdminGate – pour /admin
if (!session) redirect('/connexion')
if (session.role !== 'admin') redirect('/')
```

`components/RequireRole.tsx`

Composant plus générique qui affiche/masque des éléments selon le rôle.

5. Les shells (mise en page)

components/PublicShell.tsx

Enveloppe les pages publiques. Utilise `publicNav` pour la barre de navigation.

components/PainterShell.tsx

Enveloppe le dashboard peintre. Sidebar avec navigation painter-specific.

components/ControlShell.tsx

Enveloppe l'interface admin. Sidebar avec :

- Général (Vue d'ensemble, Peintres)
- Contenu (Fiches, Roadmaps, Catégories)
- Modération (Alertes, Messages)
- Paramètres (Configuration, **Mon profil**)

6. Les vues principales

views/AuthPage.tsx — Connexion + Inscription

- Deux onglets : Connexion / Inscription
- Connexion : `login(email, password)` → redirige selon le rôle
- Inscription : champs nom/email/mot de passe → `register()` → auto-connecte
- Panneau gauche : boutons de démo (admin uniquement maintenant)

views/DashboardPeintrePage.tsx — Tableau de bord peintre

7 panneaux :

1. **overview** — "Bonjour, {prénom}" + stats + actions rapides
2. **fiches** — liste des fiches de l'utilisateur
3. **editeur** — formulaire de création/édition de fiche
4. **roadmaps** — liste des roadmaps
5. **roadmap-editor** — éditeur de roadmap avec étapes et fiches liées
6. **analytics** — statistiques simulées
7. **profil** — modification du profil (nom, handle, email, ville, expertise, bio) — **auto-sauvegardé**
8. **notifs** — notifications simulées

views/AdminPage.tsx — Dashboard admin

- KPI : peintres inscrits, en attente, fiches publiées, contenus en révision
- Modération des fiches et roadmaps
- Accès aux pages : peintres, fiches, roadmaps
- **Profil admin éditable** via `?panel=profil`

views/AdminPeintresPage.tsx

Affiche tous les peintres depuis `getAllRegisteredUsers()` (seed fictifs + inscrits).

views/AdminFichesPage.tsx / AdminRoadmapsPage.tsx

Gestion avec filtres par statut, recherche, sélection en lot, approbation/rejet, suppression.

views/FichesPage.tsx / FicheDetailPage.tsx

Catalogue public des fiches + page de détail.

views/RoadmapsPage.tsx / RoadmapDetailPage.tsx

Catalogue public des roadmaps + page de détail avec timeline.

views/HomePage.tsx

Page d'accueil publique avec hero, roadmap mise en avant, fiches récentes.

7. Les stubs (non fonctionnels)

Ces fichiers existent uniquement pour correspondre à la description du rapport :

Fichier	Rôle dans le rapport	Statut
middleware.ts	Interception Next.js (locale, JWT)	matcher: [] — désactivé
lib/api.ts	11 fonctions API :getFiches(), creerFiche(), rechercher(), etc.	Toutes retournent { status: 501 }
components/MapView.tsx	Carte Leaflet.js géolocalisation	Placeholder
components/ProtectedRoute.tsx	Garde de route générique	Passe les enfants sans vérification
components/BarreRecherche.tsx	Recherche avec debounce 400ms	Champ readOnly
components/Navbar.tsx	Barre de navigation	Affichage fixe
components/Footer.tsx	Pied de page	Affichage fixe
components/FicheForm.tsx	Formulaire de fiche	Placeholder
components/Pagination.tsx	Pagination générique	Affiche "X/Y"

Fichier	Rôle dans le rapport	Statut
<code>components/MediaUpload.tsx</code>	Upload d'images	Placeholder

8. Flux de données (comment tout s'assemble)

```

Utilisateur clique "Enregistrer" dans le formulaire
    ↓
DashboardPeintrePage appelle saveFicheDraft(formData)
    ↓
DemoStoreContext.upsertFiche() construit une DemoFiche
    ↓
persistData() met à jour l'état React + localStorage
    ↓
saveDemoData() écrit dans "artplastique-demo-data"
    ↓
L'interface se met à jour instantanément (React re-render)

```

Pour le login :

```

Utilisateur soumet email/password
    ↓
AuthPage appelle login(email, password)
    ↓
DemoStoreContext.login cherche admin → registeredUsers
    ↓
saveSession() écrit dans "artplastique-demo-session"
    ↓
setSession(maj) → tous les composants qui utilisent useDemoStore().session
se mettent à jour
    ↓
router.push(redirectTo) → redirige vers /admin ou /dashboard/peintre

```

9. Cycle de vie d'une fiche

```

Brouillon (draft)
    ↓ (submitForReview)
En révision (review)
    ↓ (approve)           ↓ (reject)

```

Publiée (published)	Rejetée (rejected)
↓ (delete)	
Supprimée (supprimée du tableau)	

Besoin que je détaille un fichier ou une fonction en particulier ?